

MANUEL D'APPRENTISSAGE PAR LA PRATIQUE

Journal d'Audit des Reports : Enregistrement Immuable (Auteur, Dates, Motif, Écart en Jours) & Lecture Consolidée

Architecture Append-Only, Verrous PostgreSQL Physiques, Invariants Mathématiques et Moteur de Recherche Multi-Niveaux.

1. Film Mental & Objectif Sacré (Programmation Subconsciente)

Dans la théorie de la reprogrammation mentale du **Dr. Joseph Murphy**, la réussite technique découle de la clarté absolue avec laquelle l'esprit visualise le résultat final accompli. Imaginez la scène : un grand chantier de génie civil à Abidjan subit deux semaines d'intempéries torrentielles. Le Chef de Projet reprogramme les dates de coulage des fondations. Instantanément, le système consigne cet événement avec une précision chirurgicale : l'identité de l'auteur, les dates exacte avant/après, le motif officiel « Intempéries », la justification détaillée, et l'écart précis de +14 jours.

Quelques semaines plus tard, un litige contractuel surgit entre le client et l'entreprise. Un utilisateur malveillant tente de modifier la justification du report ou d'effacer la trace de ce glissement. **La base de données PostgreSQL refuse catégoriquement l'opération (RG-07)** : un trigger immuable bloque toute instruction SQL UPDATE ou DELETE avec un message d'intégrité formel. Parallèlement, le Directeur Général ouvre son tableau de bord : grâce au nouveau **Journal Consolidé des Reports**, il visualise en une seule requête l'ensemble des 27 reports survenus sur le chantier, triés par criticité, filtrables par motif et par acteur. Ressentez la paix et la certitude intérieure de concevoir un système à l'épreuve des balles juridiques et techniques.

POSTURE MENTALE DU DÉVELOPPEUR SOUVERAIN

Loi de l'Effort Inversé (Dr. Joseph Murphy) : N'essayez pas de forcer la compréhension mécanique par la tension. Ancrez simplement la conviction que l'architecture d'un journal d'audit est par nature *append-only* (ajout pur). Lorsque l'invariant d'immuabilité est posé en base de données, l'esprit est libéré de toute anxiété liée aux altérations rétroactives.

2. Architecture & Invariants Systémiques (Pólya / Dehaene / Meadows)

Suivant la méthodologie de résolution de **George Pólya**, nous décomposons le problème en invariants universels : **Données**, **Inconnue** et **Contraintes**.

Invariant	Nature & Formule	Garantie Métier / Règle de Gestion
Écart en Jours (Δt)	$\text{ecart_jours} = (\text{valeur_apres} - \text{valeur_avant}).\text{days}$	Entier relatif immuable. Positif pour un glissement/retard, négatif pour une anticipation. Persisté pour indexation SQL immédiate.
Immuabilité Physique (RG-07)	BEFORE UPDATE OR DELETE ON historique_date	Interdiction absolue de modifier ou supprimer une ligne d'audit. Refusé au niveau moteur PostgreSQL (trigger) et applicatif (save/delete).
Justification Qualifiée (RG-11)	$\text{len}(\text{justification.strip()}) \geq 30$	Obligation légale d'explication textuelle étayée d'au moins 30 caractères non-blancs.
Auteur Authentifié	auteur_id NOT NULL	Attribution systématique à l'utilisateur ayant exécuté l'action de reprogrammation.
Arborescence Consolidée	$\text{projet_id} = P \text{ OR } \text{lot_projet_id} = P \text{ OR } \text{activite_lot_projet_id} = P$	Capacité d'auditer en un point unique tous les reports intervenus sur un chantier (projet + lots + activités).

3. Guide d'Implémentation Pas-à-Pas (Le Code Maître)

L'implémentation respecte la séparation des responsabilités de Django REST Framework : **Modèle** (intégrité et contraintes), **Trigger SQL** (inviolabilité base de données), **Service** (orchestration métier transactionnelle), **Serializers** (contrats d'entrée/sortie), et **Vues DRF** (filtrage, pagination et permissions RBAC).

3.1 Évolution du Modèle HistoriqueDate (apps/projets/models/historique_date.py)

Nous enrichissons le modèle avec le champ persistant `ecart_jours` et nous surchargeons `save()` et `delete()` pour garantir l'immuabilité au niveau ORM Python.

```
# apps/projets/models/historique_date.py (Extrait clé)
class HistoriqueDate(ModeleBase):
    # ... champs existants : type_objet, projet, lot, activite, champ, valeur_avant, valeur_apres, motif, justification, auteur ...

    ecart_jours = models.IntegerField(
        _("écart en jours"),
        help_text=_("Différence en jours entre la nouvelle date et l'ancienne (valeur_apres - valeur_avant)."),
    )

    class Meta:
        db_table = "historique_date"
        verbose_name = _("historique de date")
        verbose_name_plural = _("historiques de dates")
        ordering = ["-cree_le"]
        indexes = [
            models.Index(fields=["type_objet", "champ"], name="hist_date_type_champ_idx"),
            models.Index(fields=["projet", "-cree_le"], name="hist_date_projet_idx"),
            models.Index(fields=["auteur", "-cree_le"], name="hist_date_auteur_idx"),
            models.Index(fields=["ecart_jours"], name="hist_date_ecart_idx"),
        ]

    def save(self, *args, **kwargs):
        # 1. Calcul automatique immuable de l'écart en jours
        if self.valeur_apres and self.valeur_avant:
            self.ecart_jours = (self.valeur_apres - self.valeur_avant).days
        else:
            self.ecart_jours = 0

        # 2. Règle RG-07 : Interdiction formelle de modification (append-only)
        if self.pk and HistoriqueDate.objects.filter(pk=self.pk).exists():
            raise ValidationError(_("RG-07 : L'historique d'audit des reports est strictement immuable."))

        super().save(*args, **kwargs)

    def delete(self, *args, **kwargs):
        # Règle RG-07 : Interdiction formelle de suppression
        raise ValidationError(_("RG-07 : Une entrée du journal d'audit ne peut jamais être supprimée."))
```

3.2 Trigger PostgreSQL d'Immuabilité Physique (Migration Django)

Le Socle Commun stipule : « *L'immuabilité ne se garantit pas en Python. Elle se pose en base.* » Nous créons un trigger PostgreSQL exécuté avant tout UPDATE ou DELETE sur la table `historique_date`.

```
-- Trigger PostgreSQL : Interdiction UPDATE et DELETE sur historique_date (RG-07)
CREATE OR REPLACE FUNCTION verifier_immuabilite_historique_date()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'RG-07 : La table historique_date est strictement immuable (append-only). Opération % interdite.', TG_OP;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_historique_date_immuable ON historique_date;
CREATE TRIGGER trg_historique_date_immuable
BEFORE UPDATE OR DELETE ON historique_date
FOR EACH ROW
EXECUTE FUNCTION verifier_immuabilite_historique_date();
```

3.3 Synchronisation du Service (apps/projets/services/reprogrammation.py)

Le service `reprogrammer_date_instance` transmet l'écart en jours lors de la création de chaque entrée `HistoriqueDate`, garantissant que l'opération est entièrement scellée dans la transaction atomique.

```
# Calcul de l'écart et création sécurisée dans reprogrammer_date_instance
ecart_calc = (cible_date - ancienne_date).days

HistoriqueDate.objects.create(
    type_objet=type_objet,
    champ="date_fin_prevue",
    valeur_avant=ancienne_date,
    valeur_apres=cible_date,
    ecart_jours=ecart_calc,
    motif=motif,
    justification=justif_propre,
    auteur=auteur,
    **kwargs_lien,
)
```

3.4 Serializers Enrichis (apps/projets/serializers/reprogrammation.py)

Le serializer expose toutes les dimensions décisionnelles : ecart_jours, données de l'auteur, motif complet, ainsi que le libellé et la référence de l'objet ciblé (Projet, Lot ou Activité).

```
class HistoriqueDateSerializer(serializers.ModelSerializer):
    motif = MotifReportSerializer(read_only=True)
    auteur_nom = serializers.SerializerMethodField()
    objet_libelle = serializers.SerializerMethodField()
    projet_nom = serializers.SerializerMethodField()
    projet_id = serializers.SerializerMethodField()

    class Meta:
        model = HistoriqueDate
        fields = [
            "id", "type_objet", "champ", "valeur_avant", "valeur_apres",
            "ecart_jours", "motif", "justification", "auteur_id", "auteur_nom",
            "objet_libelle", "projet_id", "projet_nom", "cree_le",
        ]

    def get_auteur_nom(self, obj: HistoriqueDate) -> str:
        if not obj.auteur:
            return "Système"
        nom_complet = f"{obj.auteur.prenom} {obj.auteur.nom}".strip()
        return nom_complet or obj.auteur.email

    def get_objet_libelle(self, obj: HistoriqueDate) -> str:
        cible = obj.activite or obj.lot or obj.projet
        return getattr(cible, "libelle", getattr(cible, "nom", str(cible)))

    def get_projet_id(self, obj: HistoriqueDate) -> str | None:
        p = obj.projet or (obj.lot.projet if obj.lot else (obj.activite.lot.projet if obj.activite else None))
        return str(p.id) if p else None

    def get_projet_nom(self, obj: HistoriqueDate) -> str | None:
        p = obj.projet or (obj.lot.projet if obj.lot else (obj.activite.lot.projet if obj.activite else None))
        return p.nom if p else None
```

3.5 Vues DRF : Journal Consolidé & Journal Global Multi-Projets

Nous mettons en place deux points d'entrée de consultation :

- 1. Journal Consolidé de Chantier** : GET /api/v1/projets/{id}/journal-reports/ — regroupe tous les décalages survenus sur le projet, ses lots et ses activités enfants.
- 2. Journal Global Multi-Projets** : GET /api/v1/projets/journal-reports/ — offre à la Direction Générale et aux Admins une vision transverse avec filtres (motif, auteur, dates, écart_min).

```
class JournalReportsFiltreMixin:
    """Mixin de filtrage commun pour le journal d'audit des reports."""
    def appliquer_filtres(self, qs, params):
        motif_id = params.get("motif_id")
        if motif_id:
            qs = qs.filter(motif_id=motif_id)
        auteur_id = params.get("auteur_id")
        if auteur_id:
            qs = qs.filter(auteur_id=auteur_id)
        type_objet = params.get("type_objet")
        if type_objet:
            qs = qs.filter(type_objet=type_objet.upper())
        ecart_min = params.get("ecart_min")
        if ecart_min is not None and ecart_min != "":
            qs = qs.filter(ecart_jours__gte=int(ecart_min))
        date_debut = params.get("date_debut")
        if date_debut:
            qs = qs.filter(cree_le__date__gte=date_debut)
        date_fin = params.get("date_fin")
        if date_fin:
            qs = qs.filter(cree_le__date__lte=date_fin)
        return qs

class ProjetJournalReportsConsolideView(APIView, JournalReportsFiltreMixin):
    """Consulte l'historique complet et consolidé des reports d'un chantier entier."""
    permission_classes = [IsAuthenticated, PermissionModule.pour(ModuleChoix.PROJETS, NiveauAcces.LECTURE), MembreDuProjet]

    def get(self, request, pk):
        projet = get_object_or_404(Projet, pk=pk)
        self.check_object_permissions(request, projet)
        qs = HistoriqueDate.objects.filter(
            models.Q(projet=projet) | models.Q(lot__projet=projet) | models.Q(activite__lot__projet=projet)
        ).select_related("motif", "auteur", "projet", "lot", "activite").order_by("-cree_le")
        qs = self.appliquer_filtres(qs, request.query_params)

        paginator = PageNumberPagination()
        page = paginator.paginate_queryset(qs, request)
        serializer = HistoriqueDateSerializer(page, many=True)
        return paginator.get_paginated_response(serializer.data)

class GlobalJournalReportsView(APIView, JournalReportsFiltreMixin):
    """Journal global transverse des reports de dates pour le DG et l'Admin."""
    permission_classes = [IsAuthenticated, PermissionModule.pour(ModuleChoix.PROJETS, NiveauAcces.LECTURE)]

    def get(self, request):
        qs = HistoriqueDate.objects.all().select_related("motif", "auteur", "projet", "lot", "activite").order_by("-cree_le")
        projet_id = request.query_params.get("projet_id")
        if projet_id:
            qs = qs.filter(models.Q(projet_id=projet_id) | models.Q(lot__projet_id=projet_id) | models.Q(activite__lot__projet_id=projet_id))
        qs = self.appliquer_filtres(qs, request.query_params)

        paginator = PageNumberPagination()
        page = paginator.paginate_queryset(qs, request)
        serializer = HistoriqueDateSerializer(page, many=True)
        return paginator.get_paginated_response(serializer.data)
```

4. Signal d'Erreur Bayésien & Pre-Mortem (Dehaene P3 / Kahneman)

Selon la théorie cognitive du **Cerveau Bayésien (Stanislas Dehaene)**, l'apprentissage s'effectue par la correction active des prédictions erronées. En ingénierie logicielle, l'exercice du **Pre-Mortem (Daniel Kahneman)** consiste à imaginer à l'avance que le système a échoué en production afin d'éradiquer les failles avant qu'elles ne se manifestent.

Scénario de Défaillance Réduit	Mécanisme de Défaillance	Parade Architecturale Implémentée
Contournement par UPDATE SQL direct	Un script d'administration ou une méthode QuerySet.update() modifie l'historique sans appeler save().	Trigger PostgreSQL natif (trg_historique_date_immuable) : lève une exception fatale au cœur du moteur SQL.
Explosion N+1 Queries sur le journal	Sérialiser 50 entrées d'audit effectue 250 requêtes SQL pour charger les auteurs, motifs, lots et projets liés.	select_related('motif', 'auteur', 'projet', 'lot', 'activite') systématique sur le QuerySet paginé.

Confusion Écart Ponctuel vs Dérive Baseline	Confondre le décalage de l'opération en cours (ex: +5 jours) et le glissement total cumulé depuis l'origine (ex: +35 jours).	ecart_jours mesure le pas ponctuel (avant vs après), tandis que jours_derive_baseline mesure l'écart à la v0.
Tentative de purge / delete accidentel	Un développeur tente un nettoyage de données de test ou une suppression en cascade.	Blocage immédiat par delete() Python (ValidationError) et blocage SQL par le trigger PostgreSQL.

5. Checklist de Tests & Protocole de Validation

Le banc de test automatisé apps/projets/tests/test_journal_audit_reports.py valide exhaustivement les exigences métier :

- **Test 1 : Calcul exact de l'écart en jours** — Vérifier que pour un report de 10 jours, ecart_jours == 10.
- **Test 2 : Immuabilité applicative Python (save & delete)** — Tenter de modifier un champ sur une instance existante ou d'appeler delete() et constater la levée de ValidationError.
- **Test 3 : Immuabilité moteur PostgreSQL** — Tenter un UPDATE brut via connection.cursor() et vérifier le blocage par le trigger SQL.
- **Test 4 : Journal Consolidé par Projet** — Créer des reports sur le projet, un lot et une activité, et vérifier qu'ils apparaissent tous dans /api/v1/projets/{id}/journal-reports/.
- **Test 5 : Journal Global & Filtrage Multi-Critères** — Filtrer par motif_id, auteur_id, et ecart_min sur /api/v1/projets/journal-reports/.
- **Test 6 : Permissions RBAC** — Vérifier qu'un membre non affecté au projet ne peut pas consulter le journal consolidé.

6. Défi Homo Docens & Rituel de Somnolence Nocturne

Le Pilier 4 de Stanislas Dehaene (Consolidation) démontre que le sommeil rejoue les circuits neuronaux fraîchement activés à une vitesse multipliée par 20. Pour ancrer définitivement ces compétences :

1. **Défi Homo Docens** : Expliquez à voix haute (ou à un collègue développeur) pourquoi une contrainte d'immuabilité purement Python est une illusion de sécurité, et comment un trigger PostgreSQL assure la garantie légale indispensable aux audits de chantiers.
2. **Rituel de Somnolence (Dr. Joseph Murphy)** : Ce soir, au moment où la somnolence vous gagne, murmurez paisiblement : « *Mon esprit intègre l'art de l'immuabilité et de la traçabilité souveraine. Mes architectures sont robustes, fiables et incorruptibles.* » Votre subconscient ordonnera les concepts pendant votre repos.